Prompt Classifier

Placeholder

November 2, 2026

---

1 Introduction

As a Linux user I sometimes find myself running command line interface (CLI) commands that I do not fully understand. Even if I look up the documentation for the commands it is not always clear what effect a command a will have on my specific machine. One alternative to parsing the documentation is to run the commands in a sandbox mirroring my system see what happens. Simple in theory but practically I ran into a problem, it was often impossible to verify whether a command worked as expected.

Consider the following questions that are among the 100 most viewed questions on `https://askubuntu.com/`:

1. How do I save terminal output to a file? (Verifiable Minimal)
2. How to list all installed packages? (Unverifiable Minimal)
3. How do I install Ubuntu alongside a pre-installed Windows with UEFI? (Unknowable)

Further lets assume that we have access to Ubuntu sandboxes, can we verify CLI commands satisfying these prompts? "How do I save the terminal output to a file?" can be verified by writing the terminal output to a file and checking that the files contents matches the terminal output. However it is not clear that the prompt "How to list all installed packages?" can be verified in a sandbox since it would require knowing all packages installed in the sandbox. Finally it is clear that no set of CLI commands run in a Ubuntu sandbox can satisfy the question "How do I install Ubuntu alongside a pre-installed Windows with UEFI?". This is a problem for any CLI command sandbox verification system since not all prompts or CLI command responses can be verified.

This project attempts to solve this problem by classifying prompts and proposed CLI command responses. The following section introduces a simple 8bit Adder system which is easier to reason about than Ubuntu and example prompts. We then outline 5 categories that we hope cover all possible system prompts and show that these prompts can be classified with a decider model like Cloudflare's Clef-Flash model. Finally we return to the three Linux prompts and show that a similar system can also be used to classify Linus prompts.

2 8bit Adder System

Lets imagine a simple 8bit adder service. The aim of this system was to be easy to reason about rather than useful. The 8bit Adders functionality is detailed below:

1. The adder is a micro service whose endpoints are accessible via routes of the form GET `http://localhost/endpoint`

2. The adder supports adding 8 bit non-negative numbers A and B at endpoint GET `http://localhost/add/{A}/{B}` and returns A+B = R if R is 8 bit or R=InvalidRequest if it is not.

3. The adder also supports getting the adder logs at endpoint GET `http://localhost/logs` which return all requests made to the service and the services responses as a json in the form request_id, timestamp, request, response.

4. All other requests to the service for the form `http://localhost/X` where X is any string including the empty string but excepting the routes detailed above return R=InvalidRequest.

3 8bit Adder Prompt Taxonomy

The prompt taxonomy I settled on for system prompts is:

- Unknowable: Prompts that cannot be answered by the system alone.
- Unverifiable: Prompts that cannot be verified by the prompter.
- Unverifiable Minimal: Prompts that cannot be verified by the prompter and whose system commands are the minimal set required.
- Verifiable: Prompts that can be verified by the prompter.
- Verifiable Minimal: Prompts that can be verified by the prompter and whose system commands are the minimal set required.

Below examples 8bit adder prompts, Spark Muse generated commands for the 8bit adder system, and my classification:

- How do I tell what language the system is written?
  Commands: GET `http://localhost/language`
  Classification: Unknowable
- How do I know the services log timestamps are accurate?
  Commands: GET `http://localhost/add/10/20`, GET `http://localhost/time`, GET `http://localhost/logs`
  Classification: Unverifiable
- How do I know the services log timestamps are accurate?
  Commands: GET `http://localhost/timestamps`
  Classification: Unverifiable Minimal
- How do I add 5 and 7 using the adder service?
  Commands: GET `http://localhost/add/5/7`, GET `http://localhost/logs`
  Classification: Verifiable
- How do I add 5 and 7 using the adder service?
  Commands: GET `http://localhost/add/5/7`
  Classification: Verifiable Minimal

4 8bit Adder Prompt Classification

The prompt classifier queries the Cloudflare decider model Clef-Flash with either 1 or 3 of the questions below following the flow in Figure 1.

- Q1: Is the prompt answerable by the system according to its specification?
- Q2: Does the prompt ask the system to perform an action with a checkable result, and not about things that require perfect knowledge of the system (such as whether the system logs are complete or accurate, what other users did, timestamp accuracy, etc.)?

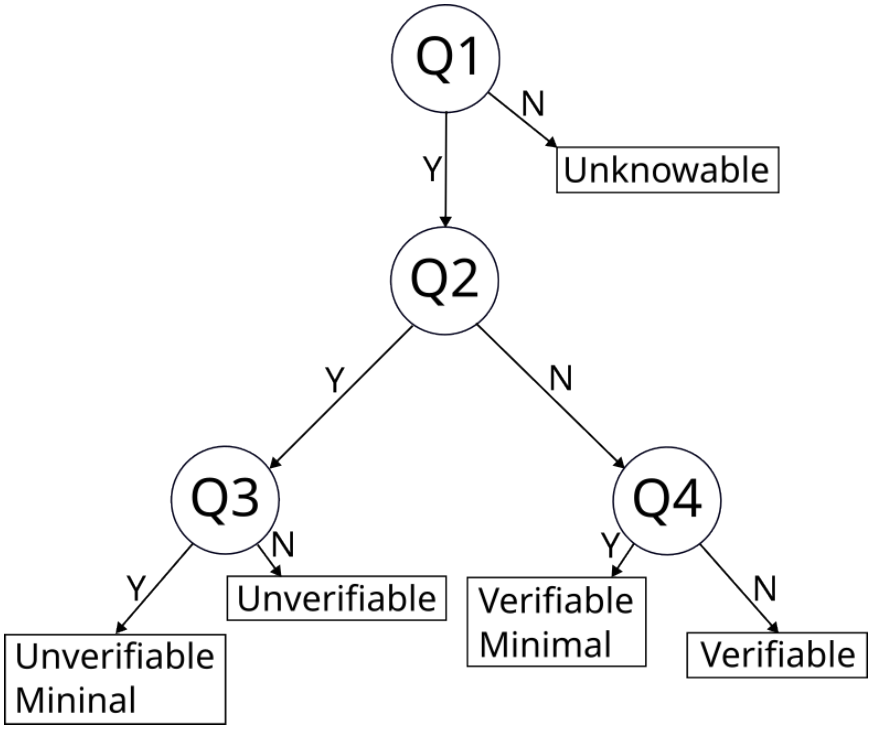


Figure 1

- Q3, Q4: Given the system specification is every request in the LLM_response required for the prompt's task?

Clef-Flash returns the answer to the question (Y/N) and a probability for the answer. With Muse Code's help I tuned both the questions and the decision cutoff points for the questions in order to achieve the following results. I think this tuning likely over fit the 8bit Adder prompt data set which consists of only 50 prompts, 10 in each category. However my goal here was not to have this prompt classification framework generalize but only to show that it can work, and I hope the following results show that.

ADD RESULTS

5 Ubuntu Prompt Classification